Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО

Отчёт

По лабораторной работе №2

Вариант: 470407

Студенты:

Долинный М. В.

Антипин Г. В.

группа *P3332*

Преподаватель:

Чупанов А. А.

г. Санкт-Петербург, 2026 г.

Описание задания

Лабораторная работа #2

Провести интеграционное тестирование программы, осуществляющей вычисление системы функций (в соответствии с вариантом).

Введите вариант: 3124123

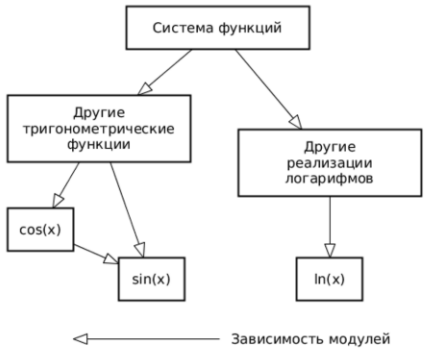
$$\begin{cases} \left(\left(\left(\left(\frac{\frac{\cot(x)}{\csc(x)}}{\sin(x)}\right) - \csc(x)\right)^3\right)^2\right) & \text{if } x \le 0 \\ \left(\frac{\left(((\log_{10}(x)+\ln(x))+\log_{10}(x)) - \left(\log_3(x)^3\right)\right)\cdot\log_{10}(x)}{\frac{\log_3(x)+((\log_5(x)-\log_2(x))\cdot\log_2(x))}{(\log_{10}(x)-\log_{10}(x))-\ln(x)}}\right) & \text{if } x > 0 \end{cases}$$

x <= 0 : (((((cot(x) / csc(x)) / sin(x)) - csc(x)) ^ 3) ^ 2)

x > 0 : (((((log_10(x) + ln(x)) + log_10(x)) - (log_3(x) ^ 3)) * log_10(x)) / ((log_3(x) + ((log_5(x) - log_2(x)) * log_2(x))) / ((log_10(x) - log_10(x)) - ln(x))))

Правила выполнения работы:

1. Все составляющие систему функции (как тригонометрические, так и логарифмические) должны быть выражены через базовые (тригонометрическая зависит от варианта; логарифмическая - натуральный логарифм).
2. Структура приложения, тестируемого в рамках лабораторной работы, должна выглядеть следующим образом (пример приведён для базовой тригонометрической функции sin(x)):

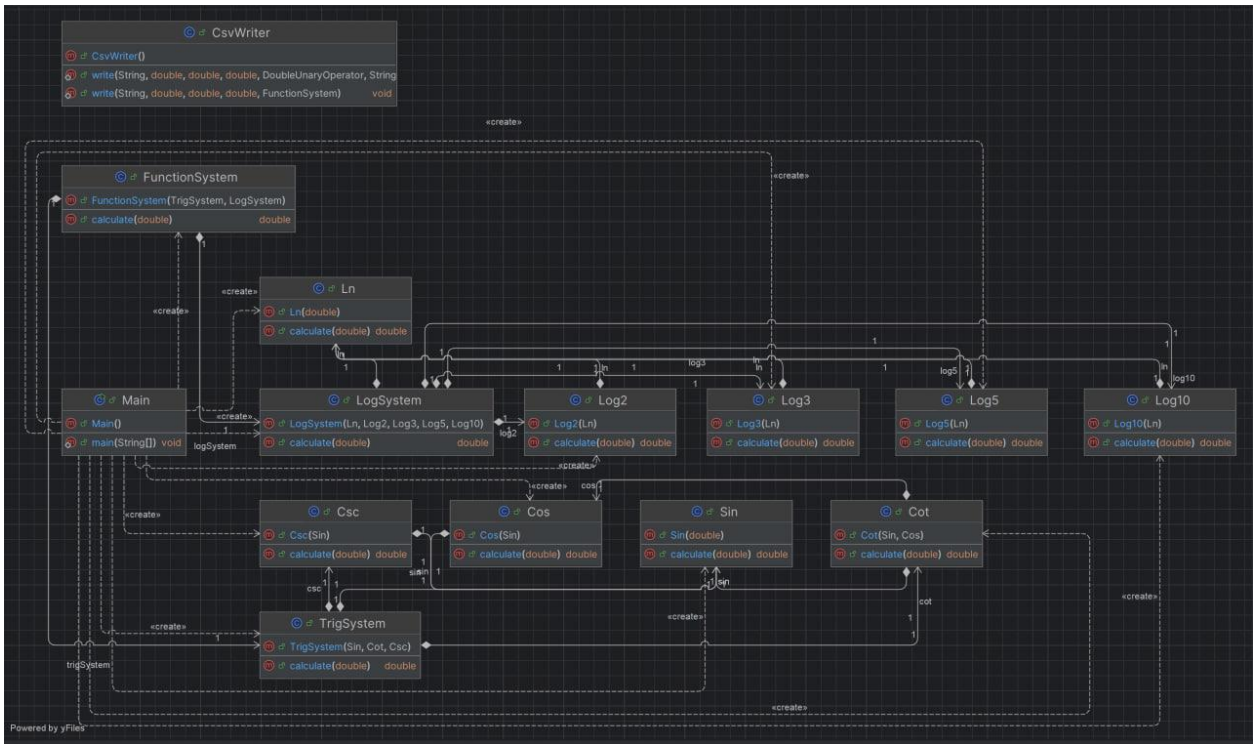


3. Обе "базовые" функции (в примере выше - sin(x) и ln(x)) должны быть реализованы при помощи разложения в ряд с задаваемой погрешностью. Использовать тригонометрические / логарифмические преобразования для упрощения функций ЗАПРЕЩЕНО.
4. Для КАЖДОГО модуля должны быть реализованы табличные заглушки. При этом, необходимо найти область допустимых значений функций, и, при необходимости, определить взаимозависимые точки в модулях.
5. Разработанное приложение должно позволять выводить значения, выдаваемое любым модулем системы, в csv файл вида «X, Результаты модуля (X)», позволяющее произвольно менять шаг наращивания X. Разделитель в файле csv можно использовать произвольный.

Порядок выполнения работы:

1. Разработать приложение, руководствуясь приведёнными выше правилами.
2. С помощью JUNIT5 разработать тестовое покрытие системы функций, проведя анализ эквивалентности и учитывая особенности системы функций. Для анализа особенностей системы функций и составляющих ее частей можно использовать сайт https://www.wolframalpha.com/.
3. Собрать приложение, состоящее из заглушек. Провести интеграцию приложения по 1 модулю, с обоснованием стратегии интеграции, проведением интеграционных тестов и контролем тестового покрытия системы функций.

UML


CsvWriter
CsvWriter()
write(String, double, double, double, DoubleUnaryOperator, String
write(String, double, double, double, FunctionSystem) void
FunctionSystem
FunctionSystem(TrigSystem, LogSystem)
calculate(double) double
Ln
Ln(double)
calculate(double) double
Main
Main()
main(String[]) void
LogSystem
LogSystem(Ln, Log2, Log3, Log5, Log10)
calculate(double) double
Log2
Log2(Ln)
calculate(double) double
Log3
Log3(Ln)
calculate(double) double
Log5
Log5(Ln)
calculate(double) double
Log10
Log10(Ln)
calculate(double) double
Csc
Csc(Sin)
calculate(double) double
Cos
Cos(Sin)
calculate(double) double
Sin
Sin(double)
calculate(double) double
Cot
Cot(Sin, Cos)
calculate(double) double
TrigSystem
TrigSystem(Sin, Cot, Csc)
calculate(double) double
«create»
logSystem
trigSystem
ln
log2
log3
log5
log10
cos
sin
csc
cot
Powered by yFiles

Описание тестового покрытия с обоснованием его выбора

Тригонометрические функции

В тригонометрическом блоке используются функции sin(x), cos(x), cot(x) и csc(x). Выбор тестов для них основан на их математических свойствах и особенностях реализации.

- Обычные значения внутри области определения: проверяются точки -1, -2, π/4, -π/4, π/3. Они нужны для проверки корректности вычислений в типичных случаях, когда функция определена и не находится в особой точке.
- Характерные точки: проверяются 0, π/2, -π/2, π. Эти значения выбраны потому, что для них результат известен аналитически, а значит, они подходят как опорные точки для проверки правильности реализации.
- Периодические значения и нормализация аргумента: дополнительно проверяются 2π и -3π/2, так как sin(x) в данной работе вычисляется через ряд Тейлора после нормализации аргумента. Такие тесты подтверждают, что реализация корректно сводит аргумент к базовому промежутку.
- Отрицательные значения: выделены отдельно, поскольку вся тригонометрическая часть системы используется именно при x <= 0. Это позволяет проверить поведение функций именно в той области, которая реально участвует в вычислении итоговой системы.
- Особые точки: для cot(x) и csc(x) критическими являются значения x = kπ, так как обе функции содержат sin(x) в знаменателе. Поэтому отдельно проверяются точки 0 и π, в которых ожидается исключение.

Такой набор тестов достаточен, потому что покрывает обычные случаи, периодичность, знаковые особенности и все ключевые точки разрыва тригонометрического блока.

Логарифмические функции

Логарифмический блок включает функции ln(x), log2(x), log3(x), log5(x) и log10(x). Их тестирование основано на свойствах логарифмов и на том, что все функции этого блока реализованы через натуральный логарифм.

- Обычные значения из области определения: используются 2, 5, 10, а также значения-степени оснований, например 4, 9, 100. Они выбраны потому, что позволяют получить точные ожидаемые результаты (1, 2) и удобно проверять корректность вычислений.

- Точка $x = 1$: проверяется отдельно, так как для всех логарифмов в этой точке результат равен 0. Это важная граничная точка, разделяющая интервалы $(0, 1)$ и $(1, +\infty)$.
- Дробные положительные значения: например, 0.5. Они нужны, потому что на интервале $0 < x < 1$ логарифмы меняют знак, и поведение функций отличается от случая $x > 1$.
- Некорректные аргументы: проверяются 0 и отрицательные значения. Это необходимо, так как логарифмические функции определены только при $x > 0$, и реализация должна корректно обрабатывать выход за область определения.
- Произвольные нетабличные значения: например, 7. Такие тесты позволяют убедиться, что реализация корректна не только на “удобных” точках, но и на обычных произвольных аргументах.

Система функций

При тестировании полной системы учитывалось, что она состоит из двух ветвей с разными условиями применимости.

- Отрицательные значения, например -1 и -2: используются для проверки того, что при $x <= 0$ система действительно переходит в тригонометрическую ветвь и корректно вычисляет результат.
- Положительные значения, например 2, 5, 10, 0.5: используются для проверки логарифмической ветви на типичных аргументах из разных частей области определения.
- Граничная точка $x = 0$: проверяется отдельно, так как это граница переключения ветвей. Она важна для проверки самого условия разбиения системы.
- Особая точка $x = 1$: проверяется отдельно, потому что в логарифмической части системы возникает неопределенность, связанная с тем, что $\ln(1) = 0$.
- Специальные опорные точки, например $x = -\pi/2$: полезны тем, что для них можно получить простой и точно известный ожидаемый результат для тригонометрической части системы.

Такое покрытие позволяет проверить не только корректность вычислений внутри каждой ветви, но и правильность маршрутизации аргумента между ними.

Тестовое покрытие

Как мы можем видеть, все работает корректно:

Test Summary

54	0	0	0.127s
tests	failures	ignored	duration

100%
successful

Packages | Classes

Package	Tests	Failures	Ignored	Duration	Success rate
ru.traphouse.functions.log	16	0	0	0.071s	100%
ru.traphouse.functions.trig	23	0	0	0.033s	100%
ru.traphouse.system	15	0	0	0.023s	100%

Generated by Gradle 8.14 at 19 мар. 2026 г., 21:16:21

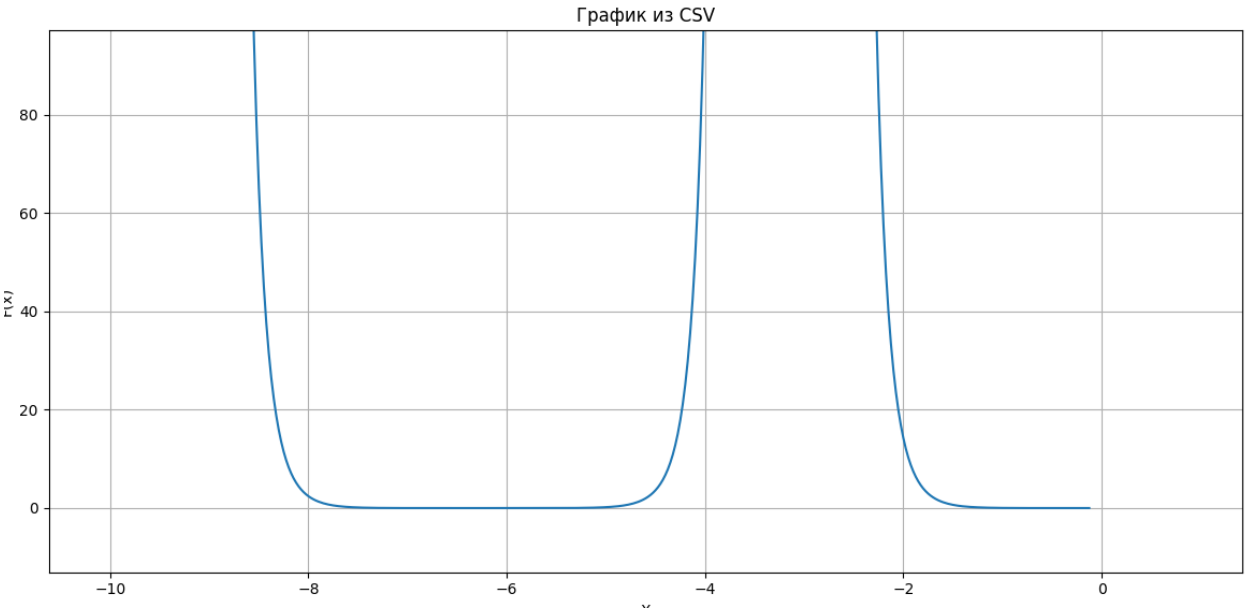
lab2

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
ru.traphouse		0 %		n/a	4	4	22	22	4	4	1	1
ru.traphouse.util		0 %		0 %	4	4	12	12	3	3	1	1
ru.traphouse.system		100 %		100 %	0	8	0	36	0	6	0	3
ru.traphouse.functions.log		100 %		100 %	0	12	0	35	0	10	0	5
ru.traphouse.functions.trig		100 %		100 %	0	13	0	33	0	8	0	4
Total	179 of 613	70 %	2 of 20	90 %	8	41	34	138	7	31	2	14

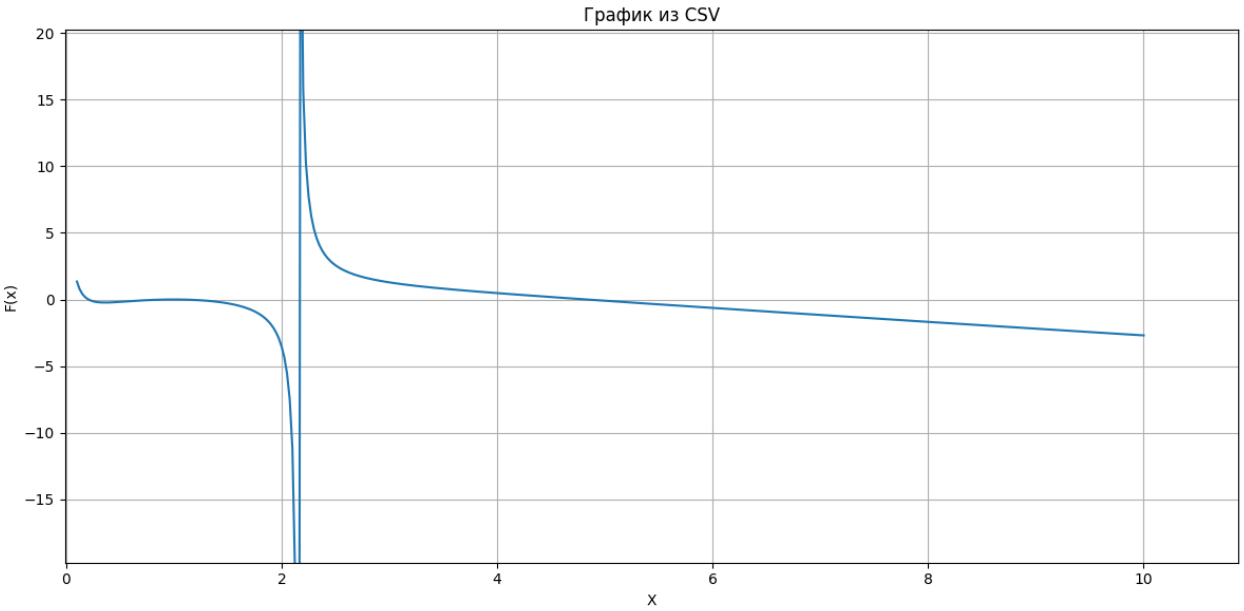
Итоговое тестовое покрытие.

Графики

trig



Log



System

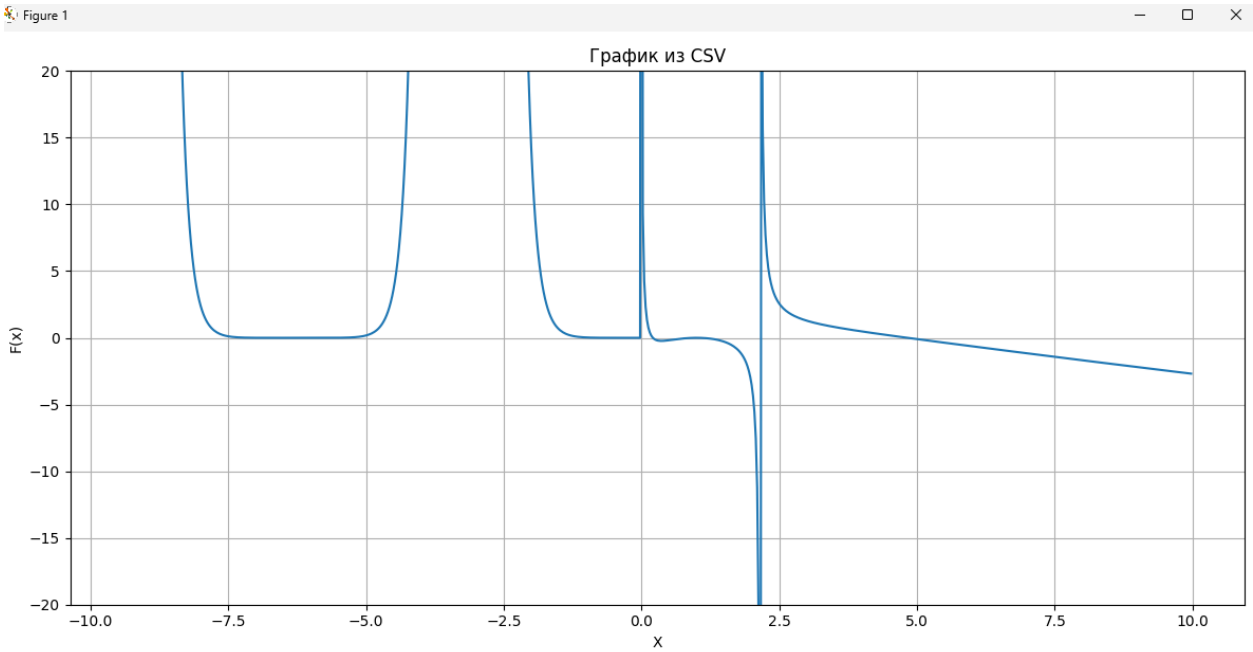


Figure 1
График из CSV
20
15
10
5
0
−5
−10
−15
−20
F(x)
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0
X

Выводы

В ходе выполнения лабораторной работы была реализована система функций,основанная на разложении в ряд. Проведено модульное и интеграционное тестирование с использованием JUnit 5 и Mockito, включая анализ граничных значений классов эквивалентности. Кроме того я научился тестировать приложения, работающие с дисплеем, получил базовые навыки CI. Выгрузка данных в CSV построенные графики подтвердили корректность вычислений. Работа позволила закрепить навыки интеграционного тестирования, работы с численными методами и проектирования модульных приложений.